Instituto Tecnológico de Costa Rica — Escuela de Ingeniería en Computadores — CE1110 Análisis de Señales Mixtas

Sistema distribuido de reducción adaptativa de ruido en tiempo real

Prototipo del sistema en alto nivel, emisor en el microcontrolador y estrategia de validación en el receptor

Saddy Guzman Rojas (2023088184) David García Cruz (2024115575)
Válerin Calderón Bran (2020033313) Luis Castro Montenegro (2013083843)

Profesor: Luis Chavarría Zamora — Segundo semestre de 2026 — 30 de octubre de 2026
Repositorio: https://github.com/SaddyGR3/sguzman_asm_2026_s2, carpeta `proyecto1/`

Resumen. Se presenta el diseño y la construcción de tres módulos del proyecto grupal[2], según el taller *FFT y sistemas de modulación*[1]: (1) el sistema completo simulado en Python con el plan de frecuencias, (2) el emisor en el microcontrolador y (3) la estrategia de validación en el receptor. La simulación incluye la cadena analógica, un filtro temporal (FIR e IIR) y uno espectral causal, sus métricas con tres semillas, la DFT contra la FFT y el costo de cada método. El emisor funciona en un ESP32 y un receptor independiente en la PC lo valida sin errores ni pérdidas.

1 Introducción

El sistema reduce ruido en tiempo real con tres ESP32 conectados por cable[2]. Este documento cubre lo que exige el taller[1]: el sistema en alto nivel con las frecuencias en que trabajará, el emisor implementado en el microcontrolador y los diagramas de la estrategia de validación en el receptor. El sistema completo en hardware corresponde a la entrega final. Como exige el enunciado del proyecto, las tablas y figuras de resultados las generan los programas de la simulación; el pie de cada tabla indica el comando que la produce.

Figura pendiente

Diagrama de bloques: PC (WAV) → entrada analógica → nodo 1 → UART1 → nodo 2 y UART2 → nodo 3 → reconstrucción → PAM8403 → parlante.

Origen: diagrama a partir de `documentos/guia_conexiones_hardware.md`

Figura 1: Arquitectura de los tres nodos y camino de la señal.

2 Arquitectura

2.1 Sistema

El sistema final usa tres ESP32 (ESP32-WROOM-32D), uno por nodo (figura 1):

- **Nodo 1, adquisición y diagnóstico:** recibe el audio de la PC por una entrada analógica (acople, divisor y filtro anti-aliasing), lo muestrea con el ADC a 24 kHz y lo lleva a 8 kHz con un FIR de decimación; quita la continua, aplica ventana y FFT y diagnostica el ruido. Envía al nodo 2 los bloques de x (INPUT) y al nodo 3 los de x y s (INPUT_REF), cada uno seguido de un paquete de diagnóstico (DIAG).
- **Nodo 2, filtrado temporal:** aplica el FIR de la banda protegida o el notch IIR, según el tipo de ruido, y reproduce el resultado por su DAC, un filtro de reconstrucción y un amplificador PAM8403.
- **Nodo 3, filtrado espectral:** aplica FFT, una ganancia por bin e IFFT; calcula las métricas de las dos rutas y las muestra en una LCD, con un LED de error y botones para cambiar de método. También tiene su salida de audio, para comparar las dos rutas de oído.

Los enlaces usan la UART1 (TX en GPIO18, al nodo 2) y la UART2 (TX en GPIO17, al nodo 3) a 921 600 baudios, con paquetes delimitados, CRC y número de secuencia; la UART0 queda para el USB y la consola. La banda protegida se configura en el nodo 1, que la reenvía a los otros nodos.

2.2 Simulación

Cada módulo de Python modela una parte del sistema (tabla 1). Para cada prueba y semilla se generan s, el ruido r y $x = s + r$; x y s pasan por la misma cadena de entrada (x_8, s_8); x_8 se filtra en el nodo 2 (y_2) y en el nodo 3 (y_3), y cada salida se compara con s_8. En esta entrega, el filtro del nodo 2 y el método del nodo 3 de cada prueba están fijados en la configuración según el ruido conocido de esa prueba (tabla 4); en la placa los sugerirá el diagnóstico del nodo 1.

3 Plan de frecuencias

Los parámetros base vienen de la Tarea 1[3] (tabla 2).

Bandas. La banda del sistema (100–3000 Hz) deja 1 kHz de transición hasta Nyquist para un filtro analógico de 2.º orden y excluye los 60 Hz de la red. La *banda*

Tabla 1: Módulos de la simulación (`proyecto1/simulacion/sim/`).

Módulo	Parte del sistema que modela
`senales.py`	PC: señales P1–P5 y archivos WAV
`analogico.py`	entrada y ADC del nodo 1; DAC y salida
`protocolo.py`	paquetes de los enlaces
`nodo2.py`	nodo 2: FIR y notch
`nodo3.py`, `metricas.py`	nodo 3: método espectral y métricas
`comparacion.py`	corrida de P1–P5 con 3 semillas
`dft.py`, `costo.py`	DFT contra FFT; costo de cada método

Tabla 2: Parámetros base del sistema.

Parámetro	Valor
f_s	8000 Hz
N, avance H	1024 ($\Delta f = 7.8$ Hz), 512 (64 ms)
Ventana	raíz de Hann periódica
Formato	float32, normalizado en $[-1, 1)$
Banda del sistema	100–3000 Hz
Enlaces	921 600 baudios, 8N1

Tabla 3: Carga por bloque de 64 ms (tamaños máximos en el cable; 10 bits por byte).

Enlace	Bytes	kbit/s	Uso
Nodo 2 (INPUT, DIAG)	2196	343	37 %
Nodo 3 (INPUT_REF, DIAG)	4252	664	72 %

protegida indica dónde está la señal útil en cada prueba; la configura el nodo 1 y la usan el FIR del nodo 2 y el método espectral del nodo 3.

Justificación conjunta. $f_s = 8$ kHz cubre la banda y limita el volumen de datos; $N = 1024$ separa los tonos de las pruebas con una ventana de 128 ms; float32 evita reescalar entre etapas y es el formato de las rutinas de ESP-DSP[7]; y 921 600 baudios alcanza para el enlace más cargado, el del nodo 3, que lleva x y s (tabla 3).

Pruebas. Cada prueba se genera con 3 semillas y 10 s, con rampas de 20 ms (tabla 4). La SNR se fija primero por potencia, $\mathrm{SNR} = 10\log_{10}\left(\sum s^2 / \sum r^2\right)$, escalando el ruido; luego s y r se escalan con el mismo factor hasta que el pico de x sea 0.8, el máximo que admite la entrada del ADC. Los WAV son estéreo: x a la izquierda y s a la derecha. P5 barre 4 niveles de agresividad para mostrar el compromiso entre quitar ruido y distorsionar la señal.

4 Cadena analógica

La simulación incluye el hardware de entrada y salida para trabajar con las atenuaciones y retardos reales.

Filtro analógico. Un Sallen-Key de ganancia unitaria (LM358) sirve de anti-aliasing y de reconstrucción:

Figura pendiente

Respuesta del Sallen-Key (10 Hz–24 kHz), del FIR de decimación (0–12 kHz) y de la salida: retención, Sallen-Key y su producto (0–4 kHz).

Origen: `resultados/figuras/cadena_analogica.png`

Figura 2: Respuesta en frecuencia de la cadena analógica.

$R_1 = R_2 = 6.8$ kΩ, $C_1 = 10$ nF y $C_2 = 4.7$ nF, con

$$H(s) = \frac{\omega_0^2}{s^2 + (\omega_0/Q)\, s + \omega_0^2}, \tag{1}$$

$$\omega_0 = \frac{1}{\sqrt{R_1 R_2 C_1 C_2}}, \qquad Q = \frac{\sqrt{R_1 R_2 C_1 C_2}}{C_2 (R_1 + R_2)}, \tag{2}$$

es decir $f_c = 3414$ Hz y $Q = 0.729$, aproximadamente Butterworth.

ADC. Con atenuación de 12 dB el ADC del ESP32 mide de 150 a 2450 mV[8]. Un acople de 10 μF y un divisor de 1.5 kΩ + 1 kΩ centran la entrada en 1.32 V, de modo que un pico de 0.8 queda entre 0.32 y 2.32 V ($x = (v - 1.32)/1.25$). El modo continuo del ESP32 no baja de 20 kHz[9], así que el nodo 1 muestrea a 24 kHz y baja a 8 kHz con un FIR de decimación de 37 coeficientes (Kaiser), calculado solo para 1 de cada 3 muestras; su retardo es de exactamente 6 muestras a 8 kHz. Sin este filtro, lo que hay entre 5 y 12 kHz se doblaría dentro de la banda útil: un tono de 7 kHz aparecería en 1 kHz.

DAC. El DAC de 8 bits retiene cada valor un periodo, lo que atenúa la parte alta de la banda y deja imágenes alrededor de 8 kHz que el filtro de reconstrucción reduce. La tabla 5 resume los resultados. Se supone una calibración ideal del ADC, que se verificará en la placa.

5 Simulación del sistema

El código está en `proyecto1/simulacion/` (tabla 1); los valores de diseño salen de `config/*.json` y 207 pruebas automáticas verifican cada módulo.

5.1 DFT directa contra FFT

La DFT se implementó desde su definición, $X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$[4], y se comparó con la FFT de NumPy para $N = 1024$ (tabla 6). Coinciden hasta el redondeo; el cociente teórico de operaciones, $N^2/((N/2)\log_2 N)$, es 204.8.

Tabla 4: Pruebas P1–P5 y método de cada ruta.

Prueba	Señal útil	Ruido	SNR	Nodo 2	Nodo 3
P1	tono de 700 Hz	tono de 2150 Hz	0 dB	notch	picos
P2	300, 700 y 1100 Hz	gaussiano 2000–2800 Hz	0 dB	FIR pasa bajas 1200 Hz	resta con protección
P3	melodía sintética	blanco 100–3000 Hz	5 dB	FIR pasa bajas 3000 Hz	resta
P4	ráfagas de 700 Hz	impulsos (Poisson)	15 dB	FIR pasa bajas 1200 Hz	resta
P5	300, 700 y 1100 Hz	gaussiano 900–1300 Hz	3 dB	notch 1100 Hz (barrido de Q)	resta (barrido de α)

Tabla 5: Cadena analógica simulada (`python -m sim.analogico`).

Magnitud	Valor
Sallen-Key a 3 kHz	-1.77 dB
Decimación: ondulación hasta 3 kHz	0.036 dB
Decimación: rechazo desde 5 kHz	49.8 dB
Alias de un tono de 7 kHz en la salida	≈ -74 dB
Salida a 3 kHz (filtro y retención)	-3.88 dB
Imagen del DAC de un tono de 300 Hz	≈ -42 dB

Tabla 6: DFT contra FFT, $N = 1024$ (medianas de 5 ejecuciones en PC; `python -m sim.dft`).

Señal	Error rel.	DFT (ms)	FFT (ms)
Tramo de P1	2.6×10^{-13}	16.13	0.020
Aleatoria	3.1×10^{-13}	16.87	0.021

Figura pendiente

$|X[k]|$ de la DFT directa y de la FFT superpuestas, y su diferencia en escala logarítmica.

Origen: `resultados/figuras/dft_fft.png`

Figura 3: DFT directa contra FFT para un tramo de P1.

5.2 Nodo 2: filtrado temporal

El nodo ejecuta $y[n] = \sum_{k=0}^{M} b_k x[n-k] - \sum_{k=1}^{N} a_k y[n-k]$ con dos candidatos.

FIR de la banda protegida. Kaiser de 37 coeficientes siempre (retardo fijo con cualquier banda), 45 dB de diseño ($\beta \approx 3.975$) y transición de 600 Hz, con los cortes en cada borde de la banda ± 300 Hz. Es pasa bajas si el borde inferior es menor que 600 Hz (todas las pruebas) y pasa banda, como diferencia de dos pasa bajas, en otro caso. Como el nodo 2 recalculará los coeficientes en la placa cada vez que cambie la banda, se implementó con la fórmula explícita (sinc por ventana, con la función de Bessel I_0 por serie) y se validó contra `scipy.signal.firwin` con error $\leq 10^{-6}$; con la banda 100–1200 Hz coincide exactamente con el FIR de la Tarea 1. Su función de transferencia,

$$H(z) = \sum_{k=0}^{36} h[k] z^{-k} = z^{-36} \sum_{k=0}^{36} h[k] z^{36-k}, \qquad (3)$$

Tabla 7: Notch en 1100 Hz del barrido de P5: radio de los polos y ganancia en los tonos útiles (`python -m sim.nodo2`).

Q	Radio	300 Hz	700 Hz
10	0.958	-0.004 dB	-0.05 dB
2.75	0.852	-0.06 dB	-0.67 dB
1	0.607	-0.46 dB	-3.81 dB

Figura pendiente

Plano z: FIR 100–1200 Hz (36 polos en el origen, ROC $|z| > 0$) y notch de P1 (ceros sobre el círculo, polos con radio 0.905, ROC $|z| > 0.905$).

Origen: `resultados/figuras/nodo2_polos_ceros.png`

Figura 4: Polos, ceros y región de convergencia causal del nodo 2.

tiene 36 polos en $z = 0$: la región de convergencia causal es $|z| > 0$ y es siempre estable. Por su simetría tiene fase lineal y un retardo fijo de 18 muestras. Rechaza 46.1 dB (100–1200 Hz) y 45.3 dB (100–3000 Hz).

Notch IIR. Con $\omega_0 = 2\pi f_0/f_s$ y $g = 1/(1 + \tan(\omega_0/2Q))$,

$$H(z) = g\,\frac{1 - 2\cos\omega_0\, z^{-1} + z^{-2}}{1 - 2g\cos\omega_0\, z^{-1} + (2g-1) z^{-2}}. \qquad (4)$$

Sus ceros están sobre el círculo unitario en $e^{\pm j\omega_0}$ (ganancia nula en f_0) y sus polos tienen radio $r = \sqrt{2g-1} < 1$: la ROC causal $|z| > r$ incluye el círculo unitario y el filtro es estable.

Q de P1. La frecuencia del notch la da la detección del nodo 1, con un error máximo de $\Delta f/2 = 3.9$ Hz. Se eligió la mayor Q (redondeada a 0.5) que atenúa al menos 30 dB con ese error sin quitar más de 0.1 dB al tono de 700 Hz: $Q = 8.5$: -30.2 dB en el peor caso, -38.2 dB con el error real de P1 (1.56 Hz), 0.013 dB de pérdida en 700 Hz y polos con radio 0.905. La tabla 7 muestra el barrido de P5.

5.3 Nodo 3: filtrado espectral causal

Parte del método de la Tarea 1[3], pero el estimador solo usa tramas actuales y pasadas, como en la placa. Tramas de 1024 con avance de 512 y raíz de Hann en análisis y síntesis (como $w[n]^2 + w[n+H]^2 = 1$, con

Figura pendiente

Magnitud de los FIR (con la línea de −45 dB), magnitud de los notch (P1 y P5 con $Q = 10$, 2.75 y 1) y fase del FIR (lineal) frente al notch.

Origen: `resultados/figuras/nodo2_respuesta.png`

Figura 5: Respuesta en frecuencia del nodo 2.

ganancia unitaria la salida reproduce la entrada); el piso de ruido es la mediana de las últimas 8 tramas, suavizada con una mediana de 31 bins. El método de *picos* atenúa 60 dB los máximos que superan 6 veces el piso fuera de la banda protegida. La *resta* aplica $G = \mathrm{clip}(1 - \alpha\,\mathrm{piso}/|X|, 0.05, 1)$[5], y la *protección de tonos* conserva los tonos estables de la banda protegida. La salida va 512 muestras (64 ms) atrasada.

5.4 Comparación de las rutas

Las métricas se calculan entre 1 y 9 s contra s_8, que pasó por la misma cadena que x_8, de modo que solo se compensa el retardo de cada ruta (18 muestras en el FIR, 0 en el notch y 512 en el nodo 3). Con y la salida de una ruta, y_s la misma ruta aplicada solo a s_8 y $r = x_8 - s_8$:

$$\mathrm{MSE} = \tfrac{1}{K}\textstyle\sum (s_8 - y)^2, \qquad \mathrm{SNR} = 10\log_{10}\frac{\sum s_8^2}{\sum (s_8 - y)^2}, \tag{5}$$

$$E_{\text{útil}} = 100\,\frac{\sum y_s^2}{\sum s_8^2}\ \%, \qquad A = 10\log_{10}\frac{\sum r^2}{\sum (y - y_s)^2}, \tag{6}$$

donde ΔSNR es la SNR de la salida menos la de x_8 y la atenuación A se calcula en total y dentro de la banda de la interferencia (tablas 8 y 9).

5.5 Costo, memoria y latencia

La tabla 10 estima cada método en multiplicaciones reales por segundo, memoria y latencia algorítmica; son modelos para anticipar la carga en el ESP32, no mediciones. La FFT hace al método espectral unas 95 veces más barato que la DFT directa por trama.

6 Emisor en el microcontrolador

El emisor (`proyecto1/firmware/nodo1/`, ESP-IDF y ESP-DSP) genera cada 64 ms un bloque de 512 muestras a 8 kHz: s con tonos de 300, 700 y 1100 Hz (amplitudes relativas 1, 0.55 y 0.30) y $x = s+$ una interferencia en 2150 Hz de amplitud 1.18, que da una SNR de 0 dB; todo con una escala común de 0.26. Los tonos se sintetizan con un acumulador de fase de 32 bits y una tabla de seno, elegida frente a `sinf()` por ser 7 veces más rápida (tabla 11); su espuria de −59 dB queda bajo el ruido de cuantización del DAC de 8 bits (≈ −48 dB). Una FFT de arranque verifica los picos en la placa.

Paquetes. Cabecera de 32 B (identificador, tipo, origen, destino, banderas, sesión, secuencia, primera muestra y tamaños), datos en float32 y CRC-32[10], codificados con COBS[6] y cerrados con un 0x00 que delimita los bloques. Al nodo 2 se envían INPUT y DIAG; al nodo 3, INPUT_REF (x y s intercalados) y DIAG. La secuencia aumenta en 1 por paquete y una bandera marca el primer paquete de cada sesión. Al arrancar, el emisor escribe un 0x00 en cada UART para que un paquete cortado por un reinicio no se una al primero de la sesión nueva.

Verificación. Prueba de lazo sin diferencias, 0 desbordes con periodo de 64 000 µs, 536–564 µs de cómputo por bloque (menos del 1 %) y 922 190 baudios medidos.

7 Estrategia de validación en el receptor

La PC actúa como receptor mediante un adaptador USB-serie (YP-05, 3.3 V) y `receptor_nodo1.py`, que aplica la misma validación que harán los nodos 2 y 3 y reutiliza el módulo de protocolo de la simulación (figuras 7–9).

- **Flujo:** corte en cada 0x00, COBS, CRC, cabecera, sesión, secuencia, orden de datos y DIAG, y energía del bloque. Los errores durante un reinicio se cuentan aparte.
- **Contenido:** FFT del tramo continuo más largo; se exigen los picos en 300, 700, 1100 y 2150 Hz (±1 Hz) con los niveles del diseño (±0.5 dB) y, en el enlace del nodo 3, una SNR de 0 ± 0.2 dB.

Cada enlace se capturó 60 s; en el del nodo 3 se reinició el emisor a los 30 s. Las capturas, guardadas en el repositorio, dan el mismo resultado al reprocesarlas (tabla 12). Tras el reinicio la sesión nueva llegó completa desde su primer paquete, con la bandera de reinicio y la primera muestra en 0, y el único error fue el paquete que el reinicio dejó cortado.

8 Discusión

Con ruido fuera de la banda útil (P1, P2) las dos rutas mejoran mucho. En P2 el FIR supera al método espectral porque su rechazo es fijo y no depende de estimar el piso. En P1 el notch se limita a unos +25 dB: anula la interferencia pero desfasa unos 2.5° el tono de 700 Hz, diferencia propia de un IIR frente a un filtro de fase lineal.

En P3 el ruido ocupa la banda de la señal y el FIR no mejora nada; la resta sí (+5.9 dB), a costa de conservar

Tabla 8: Resultados de P1–P4: media ± desviación de 3 semillas (`python -m sim.comparacion`). En P1 el notch está centrado exactamente en la interferencia y su atenuación en banda es numéricamente ilimitada.

Prueba	Ruta	SNR entrada (dB)	ΔSNR (dB)	Energía útil (%)	Aten. total (dB)	Aten. en banda (dB)
P1	nodo 2 (notch)	0.48	24.93 ± 0.00	99.7	65.9 ± 1.0	> 200
	nodo 3 (picos)		51.32 ± 0.04	100.0	51.3 ± 0.0	59.3 ± 0.0
P2	nodo 2 (FIR)	0.76	52.09 ± 0.10	99.6	57.8 ± 0.0	59.3 ± 0.1
	nodo 3 (resta con protección)		25.66 ± 0.14	99.9	25.7 ± 0.1	25.7 ± 0.2
P3	nodo 2 (FIR)	5.29	0.01 ± 0.00	99.8	0.0 ± 0.0	—
	nodo 3 (resta)		5.94 ± 0.09	60.5	19.5 ± 0.0	—
P4	nodo 2 (FIR)	15.39	3.04 ± 0.04	99.6	3.0 ± 0.0	—
	nodo 3 (resta)		6.49 ± 0.25	93.4	19.0 ± 1.1	—

Figura pendiente

Por prueba: señales alineadas en 40 ms y espectros, con el nodo 2 y el nodo 3 en paneles separados frente a s y x.

Origen: `resultados/figuras/P1.png` a `P5.png` (se elige P1 y P5 para el documento)

Figura 6: Señal limpia, contaminada y filtradas por las dos rutas.

Tabla 9: Compromiso en P5 (SNR de entrada 2.99 dB): media ± desviación de 3 semillas.

Nivel	Ruta	ΔSNR (dB)	E. útil (%)	Aten. 900–1300 Hz (dB)
Suave	$Q = 10$	1.21 ± 0.01	93.2	2.1 ± 0.0
	$\alpha = 2$	10.33 ± 0.24	93.2	18.5 ± 0.5
Medio	$Q = 2.75$	3.91 ± 0.01	89.5	7.2 ± 0.1
	$\alpha = 3$	9.53 ± 0.10	92.5	25.0 ± 0.6
Agresivo	$Q = 1$	2.52 ± 0.04	73.6	15.0 ± 0.1
	$\alpha = 8$	9.21 ± 0.02	90.8	26.0 ± 0.0
Referencia	$Q = 2.75$	3.91 ± 0.01	89.5	7.2 ± 0.1
	$\alpha = 3$, prot.	10.80 ± 0.33	97.0	15.1 ± 1.9

Tabla 10: Costo estimado (`python -m sim.costo`).

Método	Mult./s	Memoria	Latencia
FIR de decimación (nodo 1)	296 000	292 B	0.75 ms
FIR banda protegida (nodo 2)	296 000	292 B	2.25 ms
Notch IIR (nodo 2)	40 000	28 B	0
Espectral (nodo 3)	688 031	34 860 B	64 ms
DFT directa por trama	65 536 000	57 344 B	—

Tabla 11: Generador medido en la placa.

Método	Tiempo	Carga	Espuria
Tabla	501 μs	0.8 %	−59.4 dB
`sinf()`	3551 μs	5.6 %	−77.7 dB

el 61 % de la energía útil. En P5, bajar Q o subir α aumenta la atenuación del ruido pero distorsiona la señal; la protección de tonos da el mejor equilibrio (+10.8 dB y 97 % de energía útil).

El nodo 2 es barato y casi sin latencia; el nodo 3 necesita unas 2.3 veces las multiplicaciones del FIR y retarda 64 ms, pero es el único que maneja ruido superpuesto con la señal. Las dos cargas son pequeñas para un ESP32 y se medirán en la placa.

Figura pendiente

Flujo de recepción: bytes → corte en 0x00 → COBS → CRC-32 → cabecera → sesión → secuencia → datos/DIAG → energía → contadores y veredicto.

Origen: diagramas del receptor (en preparación)

Figura 7: Flujo de recepción y validación de cada paquete.

Figura pendiente

Validación del contenido: tramo continuo → ventana de Hann → FFT → 4 picos → comparación con el diseño → SNR.

Origen: diagramas del receptor (en preparación)

Figura 8: Validación del contenido espectral.

Figura pendiente

ESP32 por USB a la PC (consola); TX de GPIO18 o GPIO17 al RX del YP-05 (3.3 V, GND común) y YP-05 por USB a la PC (receptor).

Origen: diagramas del receptor (en preparación)

Figura 9: Montaje de prueba con la PC como receptor.

Tabla 12: Validación del emisor (60 s por enlace).

	Nodo 3	Nodo 2
Paquetes válidos	1855	1875
Errores y paquetes perdidos	0	0
Reinicio con EN	a los 30 s	no
Picos y niveles	exactos	exactos
SNR medida	+0.000 dB	—
Resultado	OK	OK

Los resultados son de simulación: la calibración del ADC se supone ideal, en P1 la frecuencia del notch se toma conocida (su error de detección ya se consideró al elegir Q) y las métricas usan la referencia limpia, que en la prueba final con señal desconocida puede no estar disponible.

9 Conclusiones

- La simulación reproduce el sistema completo con su plan de frecuencias y todas sus tablas se regeneran desde el repositorio.
- El emisor funciona en la placa y transmite por los dos enlaces sin errores ni pérdidas, incluso tras un reinicio.
- La estrategia de validación del receptor está implementada y probada, y es la que usarán los nodos 2 y 3.

Queda para la entrega final: circuitos analógicos, adquisición por ADC, nodos 2 y 3 en la placa, integración y pruebas P1–P6.

Reproducción

Desde `proyecto1/simulacion/`: `python -m pytest pruebas` y `python -m sim.informe`. Emisor: `receptor_nodo1.py --archivo` sobre `herramientas_pc/capturas/`.

Uso de herramientas de IA

[Completar por el equipo.]

Referencias

[1] L. Chavarría Zamora. *FFT y sistemas de modulación*. Taller, CE1110, TEC, segundo semestre de 2026.

[2] L. Chavarría Zamora. *Diseño de un sistema distribuido de reducción adaptativa de ruido en tiempo real*. Enunciado del proyecto grupal 1, versión 7, CE1110, TEC, 2026.

[3] S. Guzman Rojas, D. García Cruz, V. Calderón Bran y L. Castro Montenegro. *Caracterización espectral y prototipado de filtros para reducción de ruido*. Tarea 1, CE1110, TEC, 2026.

[4] J. G. Proakis y D. G. Manolakis. *Digital Signal Processing*, 4.ª ed. Pearson, 2007.

[5] S. F. Boll. "Suppression of acoustic noise in speech using spectral subtraction". *IEEE Trans. Acoust., Speech, Signal Process.*, 27(2):113–120, 1979.

[6] S. Cheshire y M. Baker. "Consistent overhead byte stuffing". *IEEE/ACM Trans. Netw.*, 7(2):159–172, 1999.

[7] Espressif Systems. *ESP-DSP*. https://github.com/espressif/esp-dsp.

[8] Espressif Systems. *ESP-IDF: Analog to Digital Converter (ADC)*, ESP32, v4.4. https://docs.espressif.com/projects/esp-idf/en/v4.4.8/esp32/api-reference/peripherals/adc.html.

[9] Espressif Systems. *ESP-IDF*, `components/soc/esp32/include/soc/soc_caps.h`: `SOC_ADC_SAMPLE_FREQ_THRES_LOW`. https://github.com/espressif/esp-idf.

[10] J. L. Gailly y M. Adler. *zlib manual*, función `crc32`. https://zlib.net/manual.html.